

Программирование и прототипирование ИТ-решений

Введение в программирование

Роль кода и прототипирования в работе бизнес- и системного аналитика. Среда разработки, экосистема Python и система контроля версий Git.

ЛЕКЦИЯ 1

План лекции

1 Зачем аналитику программирование

2 Прототипирование: от идеи к MVP

3 Экосистема Python

4 Среда разработки: Python + VS Code / PyCharm

5 Система контроля версий Git

6 Первая программа и следующий шаг

Роль кода в работе аналитика

01

Проверка гипотез

Быстро тестировать идеи на реальных данных, не дожидаясь ИТ-команды.

02

Автоматизация

Выгрузки, отчёты, очистка и обработка данных — без ручного копирования.

03

Общий язык

Понимать ТЗ, оценки сроков и ограничения разработчиков; говорить на языке кода.

04

Прототипы

Показывать заказчику работающий макет решения до полной разработки.

Прототипирование: от идеи к MVP

Прототип — это рабочая модель решения, которая проверяет ценность идеи до полной разработки.



POC

Proof of Concept

Проверка технической выполнимости идеи на минимальном коде.

MVP

Minimum Viable Product

Минимальный продукт с ключевой ценностью для пользователя.

ПРИЁМКА

Сценарии приёмки

Критерии готовности, по которым заказчик принимает решение.

Экосистема Python

1

Python

Язык и интерпретатор: читаемый синтаксис, быстрый старт, кроссплатформенность.

2

pip

Менеджер пакетов: установка и обновление библиотек одной командой.

3

venv

Виртуальные окружения: изоляция зависимостей для каждого проекта.

4

PyPI

Репозиторий пакетов: тысячи готовых библиотек для любых задач.

5

Библиотеки

NumPy, pandas, FastAPI, Streamlit, pytest — инструменты наших кейсов.

Среда разработки

1

Python 3.11+

Интерпретатор — выполняет ваш код.

2

VS Code / PyCharm

Редакторы кода: подсветка, отладка, автодополнение.

3

venv

Изолированное окружение проекта с зависимостями.

4

Терминал

Запуск скриптов и команд из командной строки.

5

Git

Контроль версий — встроен в редактор и терминал.

Система контроля версий Git

Git фиксирует историю изменений кода и позволяет работать над проектом в команде.

1

Репозиторий

Хранилище проекта и всей истории изменений.

2

Коммит

Зафиксированное состояние кода с описанием.

3

Ветка

Параллельная линия разработки.

4

Слияние

Объединение изменений из веток (merge).

базовые команды

```
$ git init
$ git add .
$ git commit -m "первый коммит"
$ git push origin main
$ git status    # что изменилось
```

Платформы для командной работы

- GitHub — облачные репозитории и Code Review
- GitLab — репозитории + CI/CD для автоматизации
- *В курсе Git используется на каждой лабораторной работе*

Первая программа на Python

```
my_first.py
# my_first.py – комментарий
print("Hello, world!")

name = input("Как вас зовут? ")
print(f"Привет, {name}!")
```

```
$ python my_first.py
```

– комментарий

Пояснения, которые не выполняются.

print() – вывод

Печатает текст на экран.

input() – ввод

Читает данные, которые вводит пользователь.

f-строки

Вставка значений переменных прямо в текст.

Переменная — именованная ячейка памяти. В Python тип переменной определяется автоматически (динамическая типизация).

Базовые типы данных

int

Целые числа

42, -7, 2026

float

Дробные числа

3.14, -0.5

str

Строки (текст)

"скважина", 'данные'

bool

Логический тип

True / False

динамическая типизация

```
debit = 142.5          # float
well  = "скважина №12" # str
is_active = True       # bool
type(debit)            # <class 'float'>
```

Дорожная карта курса: 2 семестра

СЕМЕСТР 1 • ОСНОВЫ PYTHON

- ✓ Синтаксис языка и типы данных
- ✓ Ветвления, циклы, алгоритмы
- ✓ Коллекции: списки, словари, множества
- ✓ Функции, модули, стандарты кода
- ✓ Файлы и обработка исключений

СЕМЕСТР 2 • ИИ И ПРОТОТИПЫ

- ✓ Структуры данных и DTO
- ✓ ИИ-инструменты: промпт-инжиниринг
- ✓ Автотестирование на pytest
- ✓ CLI-приложения и Mock API (FastAPI)
- ✓ Веб-дашборды (Streamlit) и MVP-проект

Итоги лекции

01

Код — рабочий инструмент аналитика

Проверка гипотез, автоматизация и прототипы без ожидания ИТ.

02

Прототип = PoC → MVP → продукт

Проверяем ценность решения до полной разработки.

03

Python — быстрый старт

Читаемый синтаксис и богатая экосистема библиотек.

04

Git обязателен

История изменений и совместная работа над кодом.

05

Следующий шаг — настроить окружение

Установить Python и редактор, создать venv, написать первый скрипт.